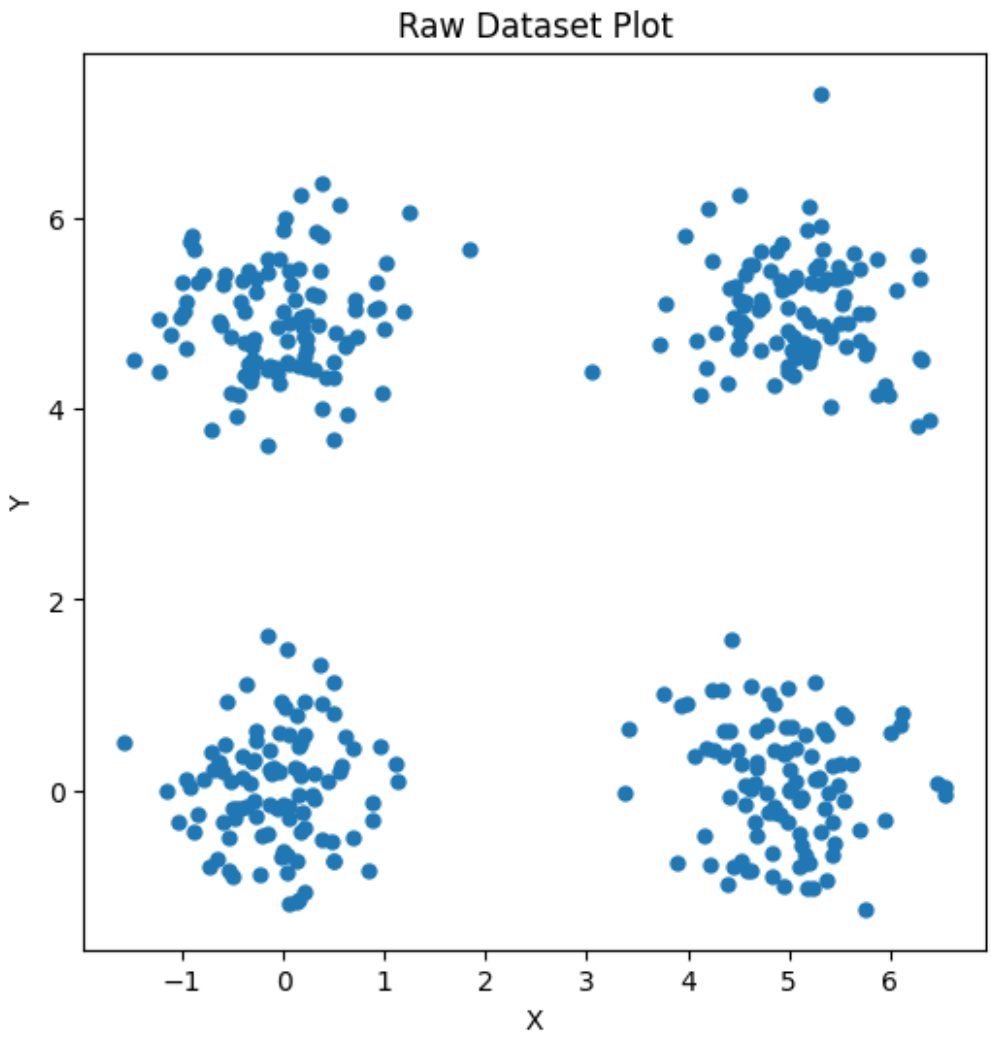
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.spatial import distance_matrix
from scipy.sparse.csgraph import minimum_spanning_tree

df = pd.read_excel("synthetic_blobs.xlsx")
X = df[["X", "Y"]].values

print(df.head())
print("Shape =", X.shape)
```

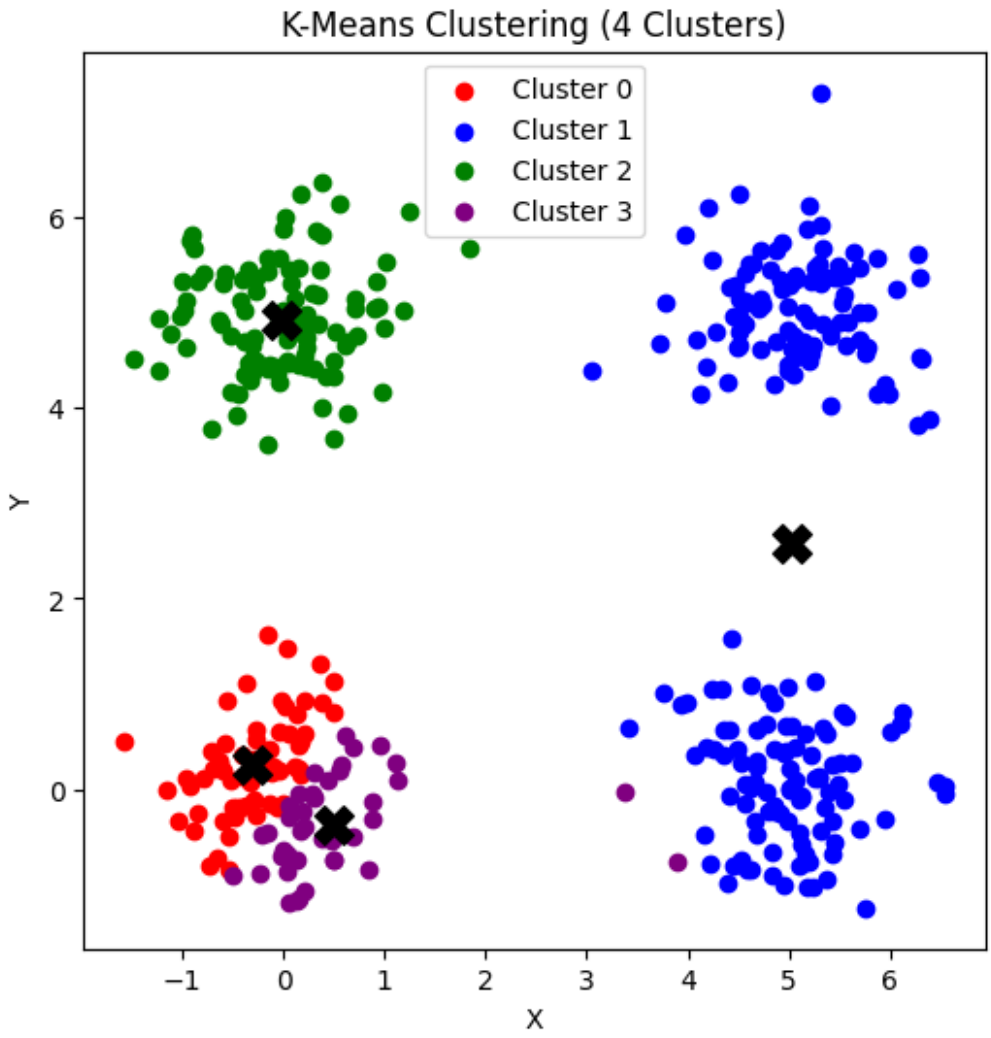
```
          X         Y  Label
0  0.298028 -0.082959      0
1  0.388613  0.913818      0
2 -0.140492 -0.140482      0
3  0.947528  0.460461      0
4 -0.281685  0.325536      0
Shape = (400, 2)
```

```python
plt.figure(figsize=(6,6))
plt.scatter(df["X"], df["Y"], s=25)
plt.title("Raw Dataset Plot")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

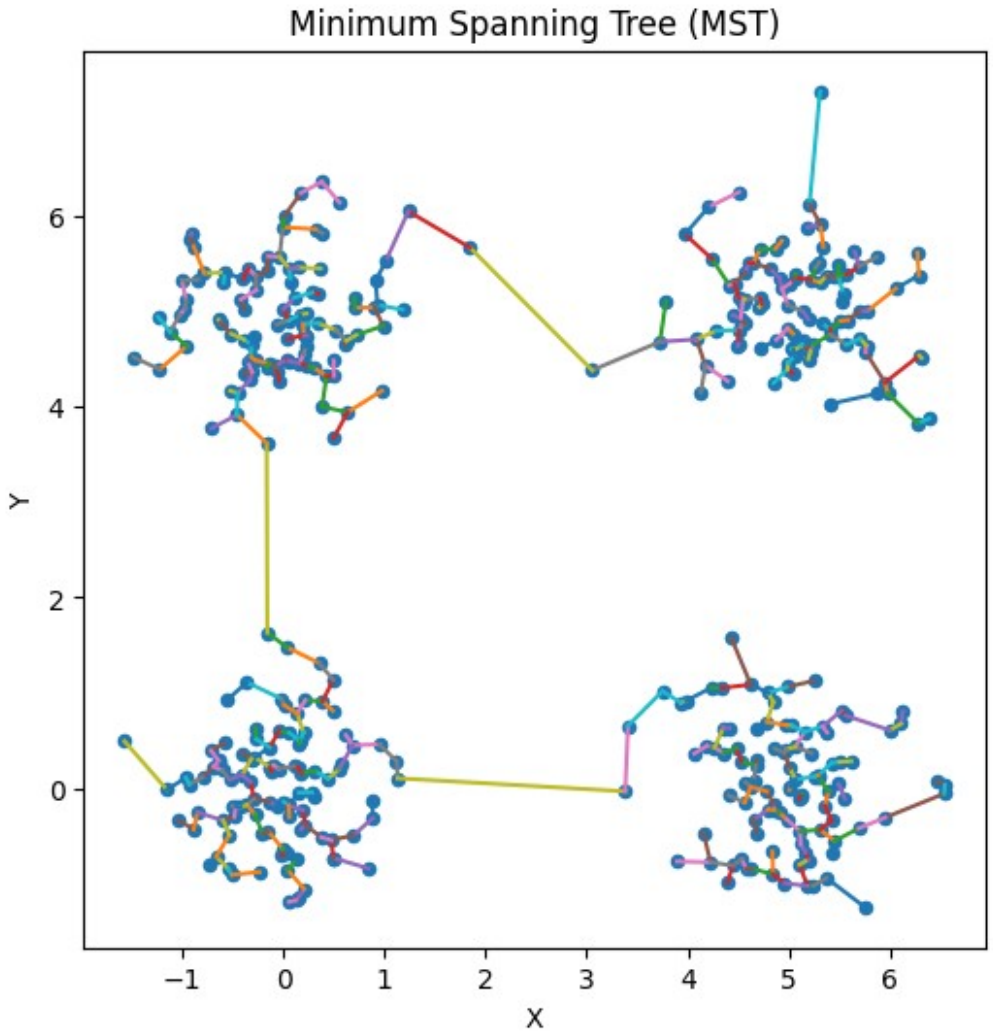



```
def kmeans_manual(X, k=4, iterations=20):
    # Step A: randomly pick centroids
    centroids = X[np.random.choice(len(X), k, replace=False)]

    for _ in range(iterations):
        # Step B: compute distances
        distances = np.linalg.norm(X[:, None] - centroids[None, :],
axis=2)

        # Step C: assign cluster to nearest centroid
        clusters = np.argmin(distances, axis=1)

        # Step D: recompute centroids
        for i in range(k):
            centroids[i] = X[clusters == i].mean(axis=0)

    return clusters, centroids
```

```
clusters, centroids = kmeans_manual(X, k=4)
plt.figure(figsize=(6,6))
colors = ["red", "blue", "green", "purple"]

for i in range(4):
    plt.scatter(X[clusters==i, 0], X[clusters==i, 1], color=colors[i],
label=f"Cluster {i}")

# centroids
plt.scatter(centroids[:,0], centroids[:,1], marker="X", s=200,
color="black")

plt.title("K-Means Clustering (4 Clusters)")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.show()
```



```
D = distance_matrix(X, X)
mst = minimum_spanning_tree(D).toarray()

plt.figure(figsize=(6,6))
plt.scatter(X[:,0], X[:,1], s=20)

# draw MST edges
for i in range(len(X)):
    for j in range(len(X)):
        if mst[i, j] != 0:
            plt.plot([X[i,0], X[j,0]], [X[i,1], X[j,1]])

plt.title("Minimum Spanning Tree (MST)")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```



```
edges = []
for i in range(len(X)):
    for j in range(len(X)):
        if mst[i, j] > 0:
            edges.append((i, j, mst[i, j]))

edges_sorted = sorted(edges, key=lambda x: x[2], reverse=True)

remove_edges = edges_sorted[:3]

mst_pruned = mst.copy()
for a, b, w in remove_edges:
    mst_pruned[a, b] = 0

adj = {i: [] for i in range(len(X))}
for i in range(len(X)):
    for j in range(len(X)):
        if mst_pruned[i, j] > 0:
            adj[i].append(j)
            adj[j].append(i)

visited = set()
cluster_assign = [-1] * len(X)
cid = 0

def dfs(node, cid):
    stack = [node]
    while stack:
        v = stack.pop()
        if v not in visited:
            visited.add(v)
            cluster_assign[v] = cid
            stack.extend(adj[v])

for i in range(len(X)):
    if i not in visited:
        dfs(i, cid)
        cid += 1

# Step F: plot clusters from MST
plt.figure(figsize=(6,6))
for k in range(cid):
    pts = X[np.array(cluster_assign) == k]
    plt.scatter(pts[:,0], pts[:,1], label=f"Cluster {k}")

# draw pruned MST edges
for i in range(len(X)):
    for j in range(len(X)):
        if mst_pruned[i, j] > 0:
            plt.plot([X[i,0], X[j,0]], [X[i,1], X[j,1]])
```

```
plt.title("MST-Based Clustering")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.show()
```